Top 40 TCS NQT 2026 Coding Questions

ARRAY QUESTIONS

1. Two Sum

Problem Statement:
Given an integer array *nums* and an integer target, return the indices of the two numbers such that they add up to target.
You may assume that each input would have exactly one solution, and you may not use the same element twice.

Example:
Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1]

Solution (Java):

```java
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> map = new HashMap<>();
        for (int i = 0; i < nums.length; i++) {
            int need = target - nums[i];
            if (map.containsKey(need)) {
                return new int[]{map.get(need), i};
            }
            map.put(nums[i], i);
        }
        return new int[]{};
    }
}
```

Time Complexity: O(n)
Space Complexity: O(n)

---

2. Best Time to Buy & Sell Stock

Problem Statement:
You are given an integer array `prices` where `prices[i]` is the price of a stock on the ith day.
You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Example:
Input: `prices = [7, 1, 5, 3, 6, 4]`
Output: 5

Solution (Java):

```java
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE, profit = 0;
        for (int p : prices) {
            minPrice = Math.min(minPrice, p);
            profit = Math.max(profit, p - minPrice);
        }
        return profit;
    }
}
```

Time Complexity: O(n)
Space Complexity: O(1)

---

3. Maximum Subarray Sum

Idea: *Kadane's Algorithm.*

```
class Solution {
    public int maxSubArray(int[] nums) {
        int cur = nums[0], best = nums[0];
        for (int i = 1; i < nums.length; i++) {
            cur = Math.max(nums[i], cur + nums[i]);
            best = Math.max(best, cur);
        }
        return best;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

4. Move Zeroes

Idea: *Two-pointer: shift non-zeros forward, fill rest with zero.*

```
class Solution {
    public void moveZeroes(int[] nums) {
        int insertPos = 0;
        for (int num : nums) {
            if (num != 0) nums[insertPos++] = num;
        }
        while (insertPos < nums.length) nums[insertPos++] = 0;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

5. Product of Array Except Self

Idea: *Prefix product from left, suffix product from right.*

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        res[0] = 1;
        for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];

        int suffix = 1;
        for (int i = n - 1; i >= 0; i--) {
            res[i] *= suffix;
            suffix *= nums[i];
        }
        return res;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1) extra (excluding output array)

---

6. Merge Two Sorted Arrays

Idea: *Fill nums1 from the back using two pointers (Two Pointers).*

```
class Solution {
    public void merge(int[] nums1, int m, int[] nums2, int n) {
        int i = m - 1, j = n - 1, k = m + n - 1;
        while (j >= 0) {
            if (i >= 0 && nums1[i] > nums2[j]) nums1[k--] = nums1[i--];
            else nums1[k--] = nums2[j--];
        }
    }
}
```

Time Complexity: O(m + n)

Space Complexity: O(1)

---

7. Find the Duplicate Number

Idea: *Floyd's Cycle Detection (treat array as linked list via indices).*

```
class Solution {
    public int findDuplicate(int[] nums) {
        int slow = nums[0], fast = nums[0];
        do {
            slow = nums[slow];
            fast = nums[nums[fast]];
        } while (slow != fast);

        slow = nums[0];
        while (slow != fast) {
            slow = nums[slow];
            fast = nums[fast];
        }
        return slow;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

8. Container With Most Water

Idea: *Two pointers from both ends, move the shorter wall inward.*

```
class Solution {
    public int maxArea(int[] height) {
        int l = 0, r = height.length - 1, best = 0;
        while (l < r) {
            int area = Math.min(height[l], height[r]) * (r - l);
            best = Math.max(best, area);
            if (height[l] < height[r]) l++;
            else r--;
        }
        return best;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

9. Trapping Rain Water

Idea: *Two pointers tracking max height seen from left and right.*

```
class Solution {
    public int trap(int[] height) {
        int l = 0, r = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;
        while (l < r) {
            if (height[l] < height[r]) {
                leftMax = Math.max(leftMax, height[l]);
                water += leftMax - height[l];
                l++;
            } else {
                rightMax = Math.max(rightMax, height[r]);
                water += rightMax - height[r];
                r--;
            }
        }
        return water;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

STRINGS

10 Java Solutions — Interview Ready

1. Valid Palindrome

Idea: *Two pointers, skip non-alphanumeric chars, compare lowercase.*

```
class Solution {
    public boolean isPalindrome(String s) {
        int l = 0, r = s.length() - 1;
        while (l < r) {
            while (l < r && !Character.isLetterOrDigit(s.charAt(l))) l++;
            while (l < r && !Character.isLetterOrDigit(s.charAt(r))) r--;
            if (Character.toLowerCase(s.charAt(l)) != Character.toLowerCase(s.charAt(r)))
                return false;
            l++; r--;
        }
        return true;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

2. Valid Anagram

Idea: *Hashing via a 26-length frequency array.*

```
class Solution {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] freq = new int[26];
        for (char c : s.toCharArray()) freq[c - 'a']++;
        for (char c : t.toCharArray()) freq[c - 'a']--;
        for (int f : freq) if (f != 0) return false;
        return true;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

3. Longest Common Prefix

Idea: *Vertical scanning column by column across all strings.*

```
class Solution {
    public String longestCommonPrefix(String[] strs) {
        if (strs.length == 0) return "";
        String prefix = strs[0];
        for (int i = 1; i < strs.length; i++) {
            while (!strs[i].startsWith(prefix)) {
                prefix = prefix.substring(0, prefix.length() - 1);
                if (prefix.isEmpty()) return "";
            }
        }
        return prefix;
    }
}
```

Time Complexity: O(n * m)

Space Complexity: O(1)

---

4. Longest Substring Without Repeating Characters

Idea: *Sliding window with a HashMap of last seen index.*

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> last = new HashMap<>();
        int start = 0, best = 0;
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            if (last.containsKey(c) && last.get(c) >= start) {
                start = last.get(c) + 1;
            }
            last.put(c, i);
            best = Math.max(best, i - start + 1);
        }
        return best;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(min(n, charset))

---

5. Group Anagrams

Idea: *HashMap keyed by sorted version of each string.*

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> map = new HashMap<>();
        for (String s : strs) {
            char[] chars = s.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
            map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(map.values());
    }
}
```

Time Complexity: O(n * k log k)

Space Complexity: O(n * k)

---

6. String Compression

Idea: *Two pointers, write char + count in place.*

```
class Solution {
    public int compress(char[] chars) {
        int write = 0, read = 0;
        while (read < chars.length) {
            char cur = chars[read];
            int count = 0;
            while (read < chars.length && chars[read] == cur) {
                read++; count++;
            }
            chars[write++] = cur;
            if (count > 1) {
                for (char c : String.valueOf(count).toCharArray())
                    chars[write++] = c;
            }
        }
        return write;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

8. Find All Anagrams in a String

Idea: *Fixed-size sliding window with frequency array comparison.*

```
class Solution {
    public List<Integer> findAnagrams(String s, String p) {
        List<Integer> res = new ArrayList<>();
        if (s.length() < p.length()) return res;

        int[] need = new int[26], window = new int[26];
        for (char c : p.toCharArray()) need[c - 'a']++;

        int k = p.length();
        for (int i = 0; i < s.length(); i++) {
            window[s.charAt(i) - 'a']++;
            if (i >= k) window[s.charAt(i - k) - 'a']--;
            if (i >= k - 1 && Arrays.equals(need, window)) res.add(i - k + 1);
        }
        return res;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

9. Isomorphic Strings

Idea: *Two HashMaps for a consistent bijective char mapping.*

```
class Solution {
    public boolean isIsomorphic(String s, String t) {
        Map<Character, Character> mapST = new HashMap<>();
        Map<Character, Character> mapTS = new HashMap<>();
        for (int i = 0; i < s.length(); i++) {
            char a = s.charAt(i), b = t.charAt(i);
            if (mapST.containsKey(a) && mapST.get(a) != b) return false;
            if (mapTS.containsKey(b) && mapTS.get(b) != a) return false;
            mapST.put(a, b);
            mapTS.put(b, a);
        }
        return true;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

LINKED LIST

10 Java Solutions — Interview Ready

Assume: class ListNode { int val; ListNode next; }

1. Reverse Linked List

Idea: *Iterative pointer reversal.*

```
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null, cur = head;
        while (cur != null) {
            ListNode next = cur.next;
            cur.next = prev;
            prev = cur;
            cur = next;
        }
        return prev;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

2. Middle of Linked List

Idea: *Slow & fast pointer technique.*

```
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

3. Linked List Cycle

Idea: *Floyd's cycle detection (slow & fast pointers).*

```
class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

4. Merge Two Sorted Lists

Idea: *Two pointers with a dummy head node.*

```
class Solution {
    public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0), cur = dummy;
        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) { cur.next = l1; l1 = l1.next; }
            else { cur.next = l2; l2 = l2.next; }
            cur = cur.next;
        }
        cur.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }
}
```

Time Complexity: O(n + m)

Space Complexity: O(1)

---

5. Remove Nth Node From End

Idea: *One-pass with two pointers, gap of n nodes.*

```
class Solution {
    public ListNode removeNthFromEnd(ListNode head, int n) {
        ListNode dummy = new ListNode(0);
        dummy.next = head;
        ListNode fast = dummy, slow = dummy;
        for (int i = 0; i < n; i++) fast = fast.next;
        while (fast.next != null) {
            fast = fast.next;
            slow = slow.next;
        }
        slow.next = slow.next.next;
        return dummy.next;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

6. Intersection of Two Linked Lists

Idea: *Two pointers; swap heads when reaching end to equalize distance.*

```
class Solution {
    public ListNode getIntersectionNode(ListNode headA, ListNode headB) {
        ListNode a = headA, b = headB;
        while (a != b) {
            a = (a == null) ? headB : a.next;
            b = (b == null) ? headA : b.next;
        }
        return a;
    }
}
```

Time Complexity: O(n + m)

Space Complexity: O(1)

---

7. Palindrome Linked List

Idea: *Find middle, reverse second half, compare both halves.*

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        ListNode slow = head, fast = head;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode prev = null;
        while (slow != null) {
            ListNode next = slow.next;
            slow.next = prev;
            prev = slow;
            slow = next;
        }

        ListNode left = head, right = prev;
        while (right != null) {
            if (left.val != right.val) return false;
            left = left.next;
            right = right.next;
        }
        return true;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

8. Add Two Numbers

Idea: *Simulate digit-by-digit addition with carry.*

```
class Solution {
    public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0), cur = dummy;
        int carry = 0;
        while (l1 != null || l2 != null || carry != 0) {
            int sum = carry;
            if (l1 != null) { sum += l1.val; l1 = l1.next; }
            if (l2 != null) { sum += l2.val; l2 = l2.next; }
            carry = sum / 10;
            cur.next = new ListNode(sum % 10);
            cur = cur.next;
        }
        return dummy.next;
    }
}
```

Time Complexity: O(max(n, m))

Space Complexity: O(max(n, m))

STACK & QUEUE

10 Java Solutions — Interview Ready

1. Valid Parentheses

Idea: *Stack: push opening brackets, match on closing brackets.*

```
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        for (char c : s.toCharArray()) {
            if (c == '(') stack.push(')');
            else if (c == '{') stack.push('}');
            else if (c == '[') stack.push(']');
            else if (stack.isEmpty() || stack.pop() != c) return false;
        }
        return stack.isEmpty();
    }
}
```

Time Complexity: O(n)

Space Complexity: O(n)

---

2. Min Stack

Idea: *Stack Design: maintain a second stack tracking running minimum.*

```
class MinStack {
    private final Deque<Integer> stack = new ArrayDeque<>();
    private final Deque<Integer> minStack = new ArrayDeque<>();

    public void push(int val) {
        stack.push(val);
        minStack.push(minStack.isEmpty() ? val : Math.min(val, minStack.peek()));
    }

    public void pop() {
        stack.pop();
        minStack.pop();
    }

    public int top() { return stack.peek(); }
    public int getMin() { return minStack.peek(); }
}
```

Time Complexity: O(1) all ops

Space Complexity: O(n)

---

3. Next Greater Element

Idea: *Monotonic decreasing stack of indices.*

```
class Solution {
    public int[] nextGreaterElements(int[] nums) {
        int n = nums.length;
        int[] res = new int[n];
        Arrays.fill(res, -1);
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && nums[stack.peek()] < nums[i]) {
                res[stack.pop()] = nums[i];
            }
            stack.push(i);
        }
        return res;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(n)

4. Daily Temperatures

Idea: *Monotonic decreasing stack storing day indices.*

```
class Solution {
    public int[] dailyTemperatures(int[] temperatures) {
        int n = temperatures.length;
        int[] res = new int[n];
        Deque<Integer> stack = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && temperatures[stack.peek()] < temperatures[i]) {
                int idx = stack.pop();
                res[idx] = i - idx;
            }
            stack.push(i);
        }
        return res;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(n)

5. Largest Rectangle in Histogram

Idea: *Monotonic increasing stack of bar indices.*

```
class Solution {
    public int largestRectangleArea(int[] heights) {
        Deque<Integer> stack = new ArrayDeque<>();
        int best = 0, n = heights.length;

        for (int i = 0; i <= n; i++) {
            int h = (i == n) ? 0 : heights[i];
            while (!stack.isEmpty() && heights[stack.peek()] >= h) {
                int height = heights[stack.pop()];
                int width = stack.isEmpty() ? i : i - stack.peek() - 1;
                best = Math.max(best, height * width);
            }
            stack.push(i);
        }
        return best;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(n)

---

6. Implement Queue using Stacks

Idea: *Two stacks: one for input, one for output (reversed order).*

```
class MyQueue {
    private final Deque<Integer> in = new ArrayDeque<>();
    private final Deque<Integer> out = new ArrayDeque<>();

    public void push(int x) { in.push(x); }

    public int pop() {
        peek();
        return out.pop();
    }

    public int peek() {
        if (out.isEmpty()) {
            while (!in.isEmpty()) out.push(in.pop());
        }
        return out.peek();
    }

    public boolean empty() { return in.isEmpty() && out.isEmpty(); }
}
```

Time Complexity: Amortized O(1)

Space Complexity: O(n)

TREES

15 Java Solutions — Interview Ready

Assume: class TreeNode { int val; TreeNode left, right; }

1. Binary Tree Inorder Traversal

Idea: *DFS: Left -> Node -> Right (iterative with explicit stack shown).*

```
class Solution {
    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode cur = root;
        while (cur != null || !stack.isEmpty()) {
            while (cur != null) {
                stack.push(cur);
                cur = cur.left;
            }
            cur = stack.pop();
            res.add(cur.val);
            cur = cur.right;
        }
        return res;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(h)

---

2. Binary Tree Preorder Traversal

Idea: *DFS: Node -> Left -> Right.*

```
class Solution {
    public List<Integer> preorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        if (root == null) return res;
        Deque<TreeNode> stack = new ArrayDeque<>();
        stack.push(root);
        while (!stack.isEmpty()) {
            TreeNode node = stack.pop();
            res.add(node.val);
            if (node.right != null) stack.push(node.right);
            if (node.left != null) stack.push(node.left);
        }
        return res;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(h)

---

3. Binary Tree Postorder Traversal

Idea: *DFS: Left -> Right -> Node, using a reversed preorder trick.*

```
class Solution {
    public List<Integer> postorderTraversal(TreeNode root) {
        LinkedList<Integer> res = new LinkedList<>();
        if (root == null) return res;
        Deque<TreeNode> stack = new ArrayDeque<>();
        stack.push(root);
        while (!stack.isEmpty()) {
            TreeNode node = stack.pop();
            res.addFirst(node.val);
            if (node.left != null) stack.push(node.left);
            if (node.right != null) stack.push(node.right);
        }
        return res;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(h)

---

4. Level Order Traversal (BFS)

Idea: *BFS using a queue, process level by level.*

```
class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        Queue<TreeNode> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            int size = queue.size();
            List<Integer> level = new ArrayList<>();
            for (int i = 0; i < size; i++) {
                TreeNode node = queue.poll();
                level.add(node.val);
                if (node.left != null) queue.add(node.left);
                if (node.right != null) queue.add(node.right);
            }
            res.add(level);
        }
        return res;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(n)

DYNAMIC PROGRAMMING

5 Java Solutions — Interview Ready

1. Climbing Stairs

Idea: *1D DP: ways(n) = ways(n-1) + ways(n-2) (Fibonacci pattern).*

```
class Solution {
    public int climbStairs(int n) {
        if (n <= 2) return n;
        int prev2 = 1, prev1 = 2;
        for (int i = 3; i <= n; i++) {
            int cur = prev1 + prev2;
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

2. House Robber

Idea: *1D DP: at each house, choose max(skip, rob + i-2).*

```
class Solution {
    public int rob(int[] nums) {
        int prev2 = 0, prev1 = 0;
        for (int num : nums) {
            int cur = Math.max(prev1, prev2 + num);
            prev2 = prev1;
            prev1 = cur;
        }
        return prev1;
    }
}
```

Time Complexity: O(n)

Space Complexity: O(1)

---

3. Longest Increasing Subsequence

Idea: *1D DP (LIS): dp[i] = longest LIS ending at index i.*

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int n = nums.length;
        int[] dp = new int[n];
        Arrays.fill(dp, 1);
        int best = 1;

        for (int i = 1; i < n; i++) {
            for (int j = 0; j < i; j++) {
                if (nums[j] < nums[i]) dp[i] = Math.max(dp[i], dp[j] + 1);
            }
            best = Math.max(best, dp[i]);
        }
        return best;
    }
}
```

Time Complexity: O(n^2)

Space Complexity: O(n)

---

4. Coin Change (Minimum Coins)

Idea: *Unbounded Knapsack: dp[amount] = min coins to make that amount.*

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;

        for (int i = 1; i <= amount; i++) {
            for (int coin : coins) {
                if (coin <= i) dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
        return dp[amount] > amount ? -1 : dp[amount];
    }
}
```

Time Complexity: O(amount * coins)

Space Complexity: O(amount)
